

Module 6: Prototyping vs Production

Speed, quality, and operational readiness

Module 6: Prototyping vs Production

Primary sources: Osmani, Chapter 6; Thomas & Hunt, Orthogonality and Reversibility.

Learning Outcomes

- Distinguish prototype and production systems.
 - Analyze tradeoffs between speed and reliability.
 - Identify gaps between generated code and production needs.
 - Explain strategies for evolving prototypes into production systems.
-

Prototype Value

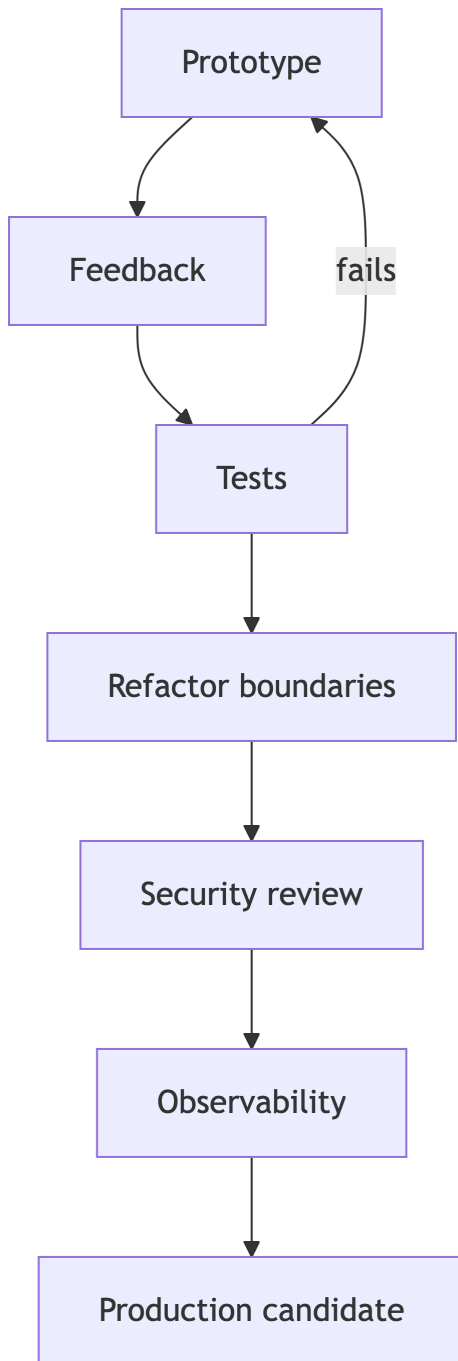
AI is strong at rapid prototyping because it can quickly create visible progress.

Prototype goals:

- Learn feasibility.
- Expose requirement gaps.
- Compare alternatives.
- Get feedback early.

Prototype risk: mistaking a learning artifact for a production system.

Prototype to Production Path



Production Means Constraints

Production-ready software must address:

- Reliability.
 - Security.
 - Observability.
 - Maintainability.
 - Dependency management.
 - Error recovery.
 - Documentation.
 - Operational cost.
-

Prototype to Production Gap

AI-generated prototypes often omit:

- Input validation.
 - Permission boundaries.
 - Logging discipline.
 - Test coverage.
 - Failure modes.
 - Deployment assumptions.
 - Data migration concerns.
-

Orthogonality

Orthogonal components can change independently.

For the Java agent project:

- Prompt construction should not depend on CLI parsing.
 - Tool dispatch should not depend on UI output.
 - Evaluation should run without live model calls.
 - Secret handling should be isolated.
-

Reversibility

Decisions should be easy to change when uncertainty is high.

Use AI to explore reversible options:

- Interfaces before implementations.
 - Adapter boundaries around model providers.
 - Configurable paths and tools.
 - Small commits that can be discarded.
-

Production Readiness Questions

- What can fail?
 - How will we know?
 - What is the safe default?
 - What should be logged?
 - What must never be logged?
 - What requires human approval?
-

Lab 3 Final Connection

Lab 3 implementation should prove that your system is controlled, not just functional.

Evidence should include:

- Required Java tools.
 - Sandbox enforcement.
 - Inspectable output.
 - Architecture notes.
 - Evaluation results.
-

Practice Activity

Take a working prototype feature and label each part:

- Keep for production.
 - Rewrite before production.
 - Add tests.
 - Add monitoring.
 - Add security review.
-

Takeaways

- AI speed is useful for learning, not a substitute for readiness.
- Orthogonal design makes generated code safer to evolve.
- Reversibility reduces the cost of AI-assisted experiments.